

RAPPORT TECHNIQUE — GÉNIE LOGICIEL

# EventManager AI

Plateforme de gestion d'événements avec intelligence artificielle

ENCADRÉ PAR

M. Chibani

RÉALISÉ PAR

Rania Kettani - Sabrine Essadeq - Houda Erimi

ANNÉE ACADÉMIQUE

2025 – 2026

DATE

Juin 2026

---

## Résumé

EventManager AI est une application web de gestion d'événements destinée aux agences et organisateurs professionnels. Le projet combine un site vitrine premium, un back-office CRUD, un tableau Kanban interactif et une génération automatique de descriptions marketing via l'API Google Gemini. L'architecture repose sur une séparation stricte *Routes* → *Services* → *Modèles* → *Base de données*, facilitant la testabilité et l'évolution. Un MVP fonctionnel a été livré en 9 Pull Requests GitHub (développement incrémental, refactoring, corrections Sonar et hotfixes), avec 34 tests automatisés.

**Mots-clés :** Flask, Kanban, Gemini, architecture en couches, qualité logicielle, GitHub.

---

# Table des matières

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                             | <b>3</b>  |
| 1.1      | Contexte . . . . .                              | 3         |
| 1.2      | Problématique . . . . .                         | 3         |
| 1.3      | Objectifs . . . . .                             | 3         |
| <b>2</b> | <b>Analyse des besoins</b>                      | <b>3</b>  |
| 2.1      | Besoins fonctionnels . . . . .                  | 3         |
| 2.2      | Besoins non fonctionnels . . . . .              | 3         |
| <b>3</b> | <b>Conception et architecture</b>               | <b>4</b>  |
| 3.1      | Architecture générale . . . . .                 | 4         |
| 3.2      | Design patterns utilisés . . . . .              | 4         |
| 3.3      | Stack technique . . . . .                       | 4         |
| <b>4</b> | <b>Réalisation</b>                              | <b>4</b>  |
| 4.1      | Site vitrine . . . . .                          | 4         |
| 4.2      | Gestion des événements . . . . .                | 8         |
| 4.3      | Tableau Kanban . . . . .                        | 9         |
| 4.4      | Génération IA avec Gemini . . . . .             | 10        |
| <b>5</b> | <b>Qualité logicielle et travaux GitHub</b>     | <b>10</b> |
| 5.1      | Workflow Git et stratégie de branches . . . . . | 10        |
| 5.2      | Historique des Pull Requests . . . . .          | 11        |
| 5.3      | Refactoring de la qualité du code . . . . .     | 11        |
| 5.4      | Corrections SonarQube . . . . .                 | 11        |
| 5.5      | Hotfixes post-MVP . . . . .                     | 12        |
| 5.6      | Tests automatisés . . . . .                     | 12        |
| <b>6</b> | <b>Résultats obtenus</b>                        | <b>12</b> |
| 6.1      | Fonctionnalités livrées et MVP . . . . .        | 12        |
| 6.2      | Synthèse des livrables GitHub . . . . .         | 12        |
| <b>7</b> | <b>Valeur métier</b>                            | <b>13</b> |
| <b>8</b> | <b>Perspectives d'amélioration</b>              | <b>13</b> |
|          | <b>Conclusion</b>                               | <b>13</b> |

# 1 Introduction

## 1.1 Contexte

La planification d'événements professionnels implique la coordination de nombreuses informations (dates, lieux, statuts, contenus promotionnels) et la rédaction répétitive de textes marketing. Les outils génériques (tableurs, notes) manquent de structure workflow et d'automatisation intelligente.

## 1.2 Problématique

Comment concevoir une solution légère, maintenable et ergonomique qui centralise le cycle de vie des événements tout en réduisant le temps de rédaction grâce à l'intelligence artificielle ?

## 1.3 Objectifs

- Livrer un MVP web couvrant CRUD événements, visualisation Kanban et génération IA.
- Appliquer une architecture en couches et des design patterns reconnus.
- Garantir la qualité via tests automatisés et revues de code (Pull Requests).
- Proposer une expérience utilisateur professionnelle (UI premium Bootstrap 5).

# 2 Analyse des besoins

## 2.1 Besoins fonctionnels

**Gestion des événements.** Création, consultation, modification et suppression d'événements (titre, date, lieu, catégorie, description, statut). Validation côté serveur et messages flash de confirmation.

**Workflow Kanban.** Six colonnes de statut : Brouillon, Planifié, Confirmé, En cours, Terminé, Annulé. Déplacement par glisser-déposer (API PATCH) avec persistance immédiate.

**Génération IA.** Production de descriptions en français à partir du contexte événement (titre, lieu, date, catégorie) via Gemini (`gemini-2.5-flash` par défaut). Intégration sur les formulaires d'ajout/édition et endpoint JSON dédié.

**Gestion des contacts.** Formulaire de contact sur le site vitrine (nom, e-mail, message) avec accusé de réception. Point d'entrée commercial ; extension CRM envisagée en perspective.

## 2.2 Besoins non fonctionnels

TABLE 1 – Besoins non fonctionnels et réponses apportées

| Exigence       | Solution retenue                                                                                                  |
|----------------|-------------------------------------------------------------------------------------------------------------------|
| Performance    | SQLite en local ; requêtes ORM indexées (date, statut, catégorie) ; API Kanban légère (JSON).                     |
| Maintenabilité | Couches découplées, services sans dépendance Flask, 34 tests pytest, documentation <code>ARCHITECTURE.md</code> . |
| Ergonomie      | Bootstrap 5, typographie premium, modal de confirmation de suppression, drag-and-drop Kanban.                     |

## 3 Conception et architecture

### 3.1 Architecture générale

Le flux applicatif suit une chaîne unidirectionnelle :

**Routes (HTTP) → Services (cas d'usage) → Modèles (ORM) → SQLite**

Les *factories* et *validators* interviennent en amont de la persistance pour construire et valider les entités.

|                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Couche présentation</b> — <code>templates/</code> , <code>static/</code> , blueprints <code>event_routes</code> , <code>landing_routes</code> |
| <b>Couche application</b> — <code>EventService</code> , <code>AIService</code> , <code>EventFactory</code> , <code>EventValidator</code>         |
| <b>Couche domaine</b> — modèle <code>Event</code> (SQLAlchemy)                                                                                   |
| <b>Infrastructure</b> — <code>config.py</code> , <code>instance/events.db</code> , API Gemini ( <code>google-genai</code> )                      |

FIGURE 1 – Architecture en couches d'EventManager AI

### 3.2 Design patterns utilisés

TABLE 2 – Design patterns et justification

| Pattern             | Intérêt                                                                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Application Factory | <code>create_app()</code> permet plusieurs configurations (dev, test, prod) et facilite les tests avec base SQLite en mémoire.                          |
| Service Layer       | <code>EventService</code> centralise CRUD, Kanban et orchestration IA ; les routes restent fines (HTTP uniquement).                                     |
| Factory Pattern     | <code>EventFactory</code> isole la construction ORM depuis les formulaires HTML.                                                                        |
| Blueprint           | Modularisation des routes ( <code>/events</code> , <code>landing</code> , <code>contact</code> ) ; enregistrement centralisé dans <code>app.py</code> . |

### 3.3 Stack technique

Python 3, Flask 3.1, Flask-SQLAlchemy, SQLite, Jinja2, Bootstrap 5, SDK `google-genai`, `pytest`.

## 4 Réalisation

### 4.1 Site vitrine

Page d'accueil premium (`landing.html`) : sections marketing, navigation ancrée, formulaire de contact POST vers `/contact`. Layout dédié (`layouts/premium.html`) avec typographie Cormorant Garamond / Outfit.



FIGURE 2 – Page d'accueil — hero section avec navigation



FIGURE 3 – Page d'accueil — CTA et image mise en valeur



FIGURE 4 – Section à propos — présentation de la plateforme

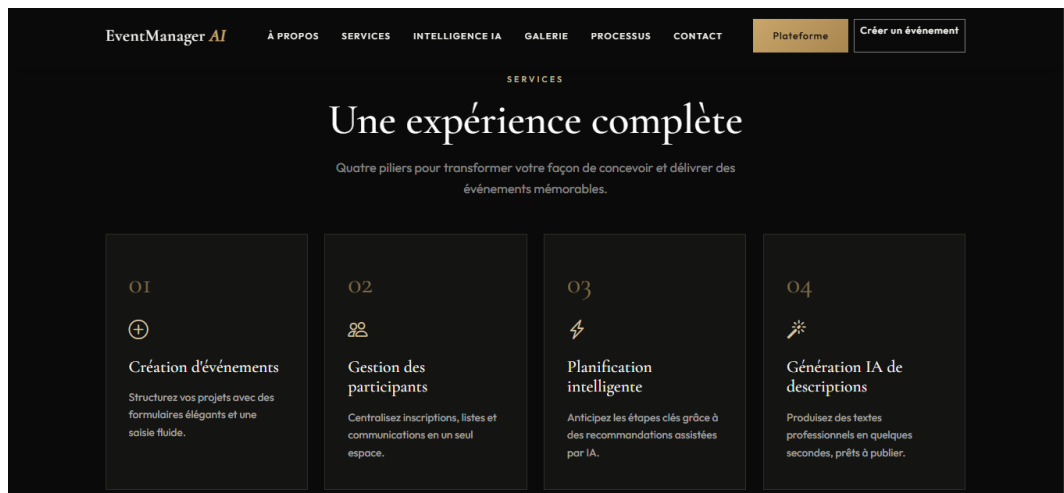


FIGURE 5 – Section services — quatre piliers de la plateforme

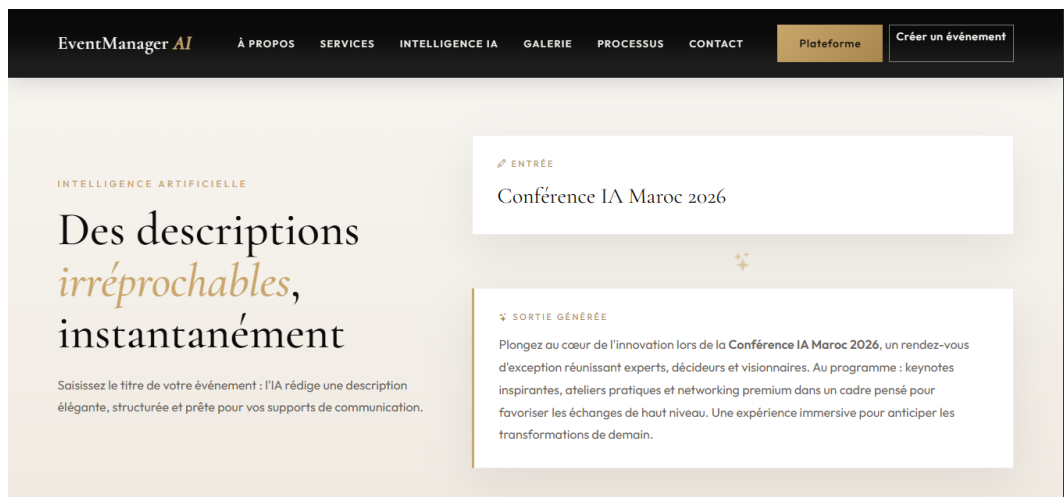


FIGURE 6 – Section Intelligence IA — génération de descriptions



FIGURE 7 – Section processus — quatre étapes et indicateurs

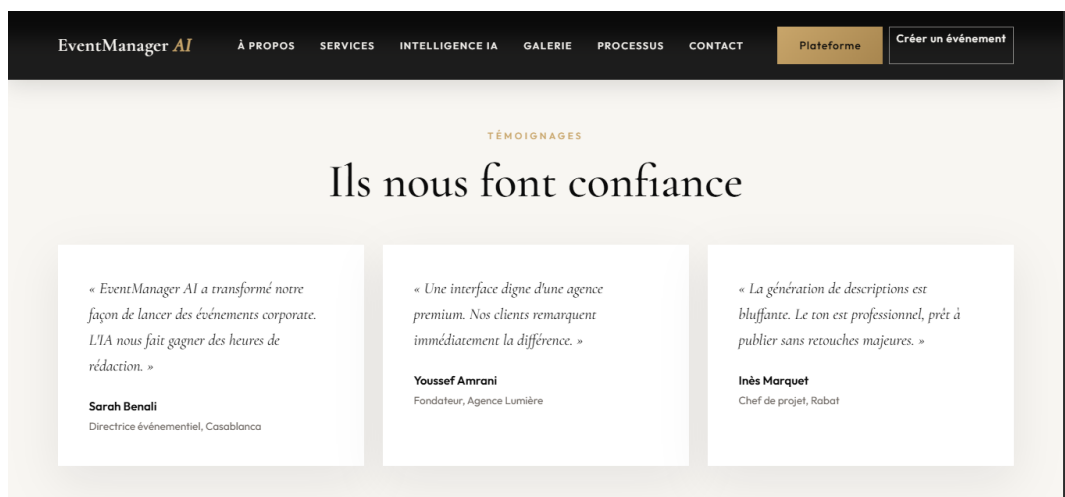


FIGURE 8 – Section témoignages — avis clients



EventManager **AI** À PROPOS SERVICES INTELLIGENCE IA GALERIE PROCESSUS CONTACT Plateforme Créer un événement

CONTACT

## Échangeons sur votre prochain événement

Une question, une démonstration ou un partenariat ? Notre équipe vous répond sous 48 h.

✉ contact@eventmanager.ai

📍 Maroc · Remote worldwide

NOM COMPLET

E-MAIL

MESSAGE

Envoyer le message

FIGURE 9 – Section contact — formulaire de prise de contact

EventManager **AI**

Plateforme SaaS premium de gestion d'événements augmentée par l'intelligence artificielle.

LinkedIn Instagram X

Plateforme

- Mes événements
- Créer un événement
- Génération IA

Découvrir

- À propos
- Services
- Processus

Newsletter

Recevez nos actualités produit et conseils événementiels.

Votre e-mail

S'inscrire

© 2026 EventManager AI. Tous droits réservés.

Conçu pour l'excellence événementielle.

FIGURE 10 – Footer — liens, réseaux sociaux et newsletter

## 4.2 Gestion des événements

Routes `GET/POST /events/add` et `/events/<id>/edit`. Validation via `EventValidator`; persistance via `EventService.create_from_form / update_from_form`. Suppression avec modal Bootstrap 5 (remplacement de `window.confirm`) et `POST /events/<id>/delete`.

EventManager *AI*    TABLEAU KANBAN    CRÉER UN ÉVÉNEMENT    Site vitrine

PLATEFORME

## Créer un événement

Renseignez les informations et choisissez le statut Kanban de départ.

TITRE \*

STATUT \*

Brouillon

LIEU \*

FIGURE 11 – Formulaire de création d'événement — partie supérieure

EventManager *AI*    TABLEAU KANBAN    CRÉER UN ÉVÉNEMENT    Site vitrine

CATÉGORIE \*

DATE \*

mm/dd/yyyy

DESCRIPTION \*

+ GÉNÉRER LA DESCRIPTION

① Ajoutez `GENINI_APT_KEY` dans votre fichier `.env` pour activer l'IA.

\* Tous les champs sont obligatoires. Vous pourrez aussi déplacer l'événement sur le tableau Kanban.

Enregistrer l'événement    Annuler

FIGURE 12 – Formulaire de création d'événement — champs description et bouton IA

### 4.3 Tableau Kanban

Vue `/events/` : colonnes par statut, cartes événement (`kanban_card.html`). JavaScript (`kanban.js`) : drag-and-drop, appel `PATCH /events/<id>/status`, mise à jour visuelle du badge statut.

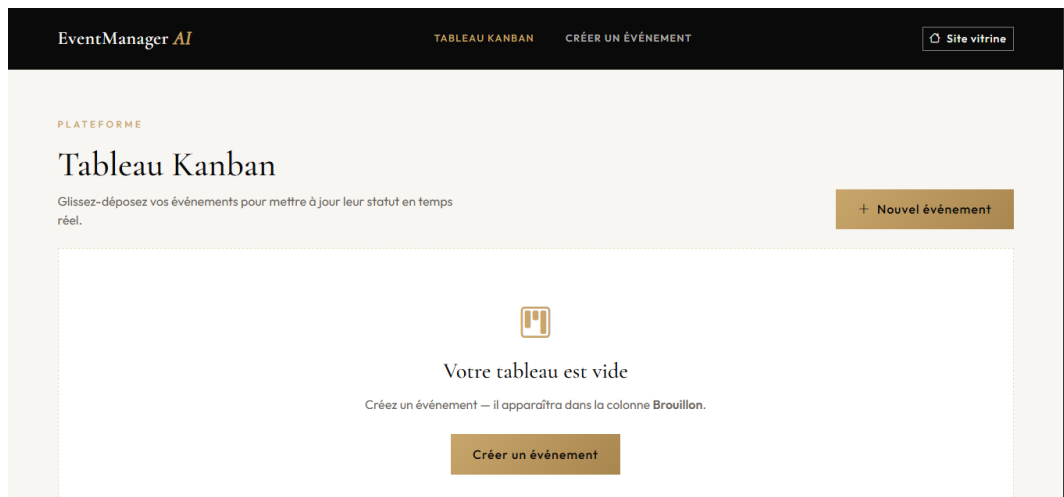


FIGURE 13 – Tableau Kanban — vue plateforme avec état vide

#### 4.4 Génération IA avec Gemini

`AIService` encapsule l'appel `genai.Client().models.generate_content`. Prompt structuré en français, ton premium, 2–4 paragraphes. Endpoints : `POST /events/ai/generate-description` (formulaire) et `POST /events/<id>/generate-description` (persistance). Clé API et modèle configurables via `.env` (`GEMINI_API_KEY`, `GEMINI_MODEL`).

La section Intelligence IA du site vitrine (figure 5 ci-dessus) illustre le résultat : à partir du simple titre *Conférence IA Maroc 2026*, Gemini génère un texte marketing professionnel prêt à publier.

## 5 Qualité logicielle et travaux GitHub

Le dépôt GitHub (`rania-kett/EventManagerAI`) a structuré le projet autour d'un développement incrémental : chaque fonctionnalité livrée via une branche `feature/*`, fusionnée dans `develop` puis `main` après revue (Pull Request).

### 5.1 Workflow Git et stratégie de branches

- `main` — branche stable, reflète le MVP livré.
- `develop` — intégration des features avant promotion vers `main`.
- `feature/*` — une branche par capacité métier (CRUD, Kanban, IA).
- `hotfix/*` — correctifs ciblés fusionnés directement (UX, i18n).
- **Conventional Commits** — préfixes `feat:`, `fix:`, `refactor:` pour traçabilité.

## 5.2 Historique des Pull Requests

TABLE 3 – Chronologie des Pull Requests GitHub (9 fusions)

| PR  | Branche                    | Livrable principal                                                            |
|-----|----------------------------|-------------------------------------------------------------------------------|
| #2  | feature/add-event          | Site vitrine premium, formulaire création, EventValidator, layouts Bootstrap. |
| #3  | feature/edit-event         | Édition événement, pré-remplissage formulaire, 4 tests dédiés.                |
| #4  | feature/delete-event       | Suppression POST, confirmation utilisateur, flash message.                    |
| #6  | feature/list-events        | Tableau Kanban (6 colonnes), drag-and-drop, kanban.js, statuts ORM.           |
| #7  | feature/ai-description     | Intégration Gemini, AIService, ai-generate.js, endpoints JSON.                |
| #8  | feature/sonar-improvements | Refactoring majeur + résolution issues SonarQube (voir 5.3).                  |
| #9  | hotfix/status-language     | Traduction libellés Kanban en français (Brouillon, Confirmé...).              |
| #10 | hotfix/delete-modal        | Modal Bootstrap 5, migration SDK google-genai, correctif Windows Flask.       |

## 5.3 Refactoring de la qualité du code

La PR #8 regroupe deux commits structurants (`refactor: improve code quality`, `fix: resolve sonar issues`), soit 26 fichiers modifiés (~600 lignes refactorisées).

**Extraction de responsabilités.**

- `routes/ai_handlers.py` — logique JSON IA sortie des routes HTTP.
- `services/messages.py` — messages flash centralisés (`flash_event_created`, etc.).
- `constants/event_form.py` — champs et libellés formulaire (DRY templates/tests).
- `utils/dates.py` — parsing dates isolé du validateur et de la factory.
- `templates/partials/event_form_fields.html` — partial Jinja2 partagé add/edit.

**Simplification des routes.** La fonction `_handle_event_form()` unifie les flux GET/POST création et édition ; `event_routes.py` passe de handlers dupliqués à une délégation systématique vers `EventService`.

**Factorisation des tests.** `tests/conftest.py` enrichi (fixtures `client`, `sample_event`) ; suppression de code boilerplate dans `test_add_event`, `test_edit_event`, `test_delete_event`.

## 5.4 Corrections SonarQube

TABLE 4 – Corrections issues SonarQube

| Fichier                             | Correction                                                                                      |
|-------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>config.py</code>              | Constante <code>DEFAULT_GEMINI_MODEL</code> , factorisation <code>is_gemini_configured</code> . |
| <code>models/event.py</code>        | Suppression code mort et imports inutilisés.                                                    |
| <code>services/ai_service.py</code> | Simplification gestion erreurs API, lisibilité.                                                 |
| <code>static/js/kanban.js</code>    | Variables <code>const/let</code> , portée réduite.                                              |
| <code>static/js/landing.js</code>   | Même traitement ; conformité règles JS Sonar.                                                   |
| <code>utils/dates.py</code>         | Gestion explicite des cas limites de parsing.                                                   |

## 5.5 Hotfixes post-MVP

**PR #9 — Internationalisation Kanban.** Libellés de statut passés de l’anglais au français dans `event_status.py`.

**PR #10 — UX suppression et stack IA.** Remplacement de `window.confirm()` par une modal Bootstrap 5 professionnelle. Migration `google-generativeai` → `google-genai` (SDK officiel GA). Correctif `app.py` : désactivation du reloader Flask sous Windows.

## 5.6 Tests automatisés

34 tests pytest couvrant modèle, CRUD, Kanban, IA (mock Gemini), landing et contact.

TABLE 5 – Répartition des tests automatisés

| Module                                             | Tests     |
|----------------------------------------------------|-----------|
| Ajout événement ( <code>test_add_event</code> )    | 5         |
| Édition ( <code>test_edit_event</code> )           | 4         |
| Suppression ( <code>test_delete_event</code> )     | 3         |
| Kanban ( <code>test_kanban</code> )                | 6         |
| Service IA ( <code>test_ai_service</code> )        | 6         |
| Modèle / factory ( <code>test_event_model</code> ) | 5         |
| Application / landing                              | 5         |
| <b>Total</b>                                       | <b>34</b> |

## 6 Résultats obtenus

### 6.1 Fonctionnalités livrées et MVP

Le MVP couvre l’intégralité du parcours organisateur : vitrine → création → suivi Kanban → édition → description IA → suppression confirmée.

### 6.2 Synthèse des livrables GitHub

TABLE 6 – Indicateurs du dépôt GitHub

| Indicateur                       | Valeur                                            |
|----------------------------------|---------------------------------------------------|
| Pull Requests fusionnées         | 9                                                 |
| Branches <code>feature/</code>   | 5                                                 |
| Branches <code>hotfix/</code>    | 2                                                 |
| Commits de refactoring / qualité | 2 ( <code>18e13bb</code> , <code>0e97e28</code> ) |
| Tests automatisés (pytest)       | 34                                                |
| Fichiers Python (hors tests)     | ~25                                               |

TABLE 7 – Tableau récapitulatif des fonctionnalités

| Fonctionnalité               | MVP | Tests | Remarque                             |
|------------------------------|-----|-------|--------------------------------------|
| Site vitrine premium         | ✓   | ✓     | Sections + contact                   |
| CRUD événements              | ✓   | ✓     | Validation serveur                   |
| Tableau Kanban (6 statuts)   | ✓   | ✓     | Drag-and-drop                        |
| Génération description IA    | ✓   | ✓     | Gemini via <code>google-genai</code> |
| Modal suppression Boots-trap | ✓   | ✓     | UX professionnelle                   |
| Formulaire contact           | ✓   | ✓     | Accusé de réception                  |
| Authentification             | –   | –     | Perspective                          |
| Multi-utilisateurs           | –   | –     | Perspective                          |

## 7 Valeur métier

**Gains de temps.** La génération IA réduit la rédaction manuelle de descriptions (estimation : 15–30 min par événement évitées).

**Productivité.** Le Kanban visualise l’avancement sans tableur externe ; un seul outil remplace notes + suivi statut.

**Automatisation IA.** Contenu marketing cohérent (ton premium, français) à partir de méta-données structurées.

**Potentiel commercial.** Site vitrine intégré, formulaire contact, base extensible vers offre SaaS pour agences événementielles.

## 8 Perspectives d’amélioration

- **Authentification** — comptes organisateurs, sessions sécurisées.
- **Multi-utilisateurs** — rôles (admin, planner, lecteur), filtrage par équipe.
- **Dashboard analytique** — KPIs (événements par statut, taux de conversion contact).
- **Déploiement cloud** — conteneurisation (Docker), WSGI (Gunicorn), PostgreSQL managé.
- **SaaS** — multi-tenant, facturation, quotas API Gemini par client.

## Conclusion

EventManager AI démontre la faisabilité d’une plateforme événementielle moderne avec une stack Python légère et une intégration IA maîtrisée. Le cycle de vie GitHub — 9 Pull Requests, refactoring structurant (PR #8), corrections Sonar et hotfixes UX — illustre une démarche qualité compatible avec un contexte professionnel. Les choix architecturaux (factory, services, blueprints) ont permis ce développement incrémental et une couverture de tests solide pour un MVP. Les prochaines étapes prioritaires concernent l’authentification, le déploiement production et la montée en charge vers un modèle SaaS.

## Références

- [1] P. Grinberg, *Flask Web Development*, O’Reilly, 2018.
- [2] R. C. Martin, *Clean Architecture*, Prentice Hall, 2017.

- [3] Google, *Gemini API — Google GenAI SDK for Python*, documentation officielle, 2025. <https://ai.google.dev/gemini-api/docs>
- [4] Bootstrap Team, *Bootstrap 5 Documentation*, 2023. <https://getbootstrap.com/docs/5.3/>
- [5] H. Krekel *et al.*, *pytest*, framework de tests Python, 2024. <https://docs.pytest.org/>